

Rapidly Build and Deploy a Full-Stack App with Cursor

Advanced AI-assisted SaaS application development

Lucas Soares

September 2026

Table of Contents

- 1. Cursor & Context Management**
- 2. Building Your First Full App**
- 3. Advanced App Development & Architecture**
- 4. Real-World Full-Stack Application**
- 5. Deployment, Testing & Production Best Practices**

Cursor 3 & Context Management

AI-First Code Editor for Production Development

Not just autocomplete—full stack development

AI-First Code Editor for Production Development

Not just autocomplete—full stack development

Composer 2.5

Standard + Fast tiers—an iteration over Composer 2

Agents Window

Parallel agents across local, worktree, cloud & SSH

Bugbot

Automated PR review on Composer 2.5—~90-second reviews

MCP Integration

Connect to GitHub, Figma, databases

[1][Cursor 3 Changelog](#)

Core Features

Chat (Cmd+L)

- Multi-tab conversations
- Context checkpoints
- Quick Q&A

Tab Autocomplete

- Multi-line predictions
- Context-aware
- Pattern learning

Agent (Cmd+I)

- Multi-file editing
- Autonomous execution
- Planning + building

Inline Edit (Cmd+K)

- In-place modifications
- Natural language
- Stay in flow

The Three-Step Framework



1. EXPLORE

Share context
Identify files
Discuss options



2. PLAN

Step-by-step plan
Markdown checklists
Break down tasks



3. BUILD

Execute iteratively
Review each step
Commit regularly

The Three-Step Framework



1. EXPLORE

Share context
Identify files
Discuss options



2. PLAN

Step-by-step plan
Markdown checklists
Break down tasks



3. BUILD

Execute iteratively
Review each step
Commit regularly



This framework prevents context overload and ensures quality output

^[1][Cursor for Staff Engineers](#)

Context Management: The Foundation



Key Challenge

Context management is like telling a story—when it gets convoluted, the AI loses track

Context Management: The Foundation



Key Challenge

Context management is like telling a story—when it gets convoluted, the AI loses track

Why It Matters

- Fixed context windows (128k-1M tokens)
- Comprehension $\neq$ token count
- Poor context = hallucinations

Best Practices

- Use @-mentions for precision
- Start new chats frequently
- Configure `.cursorignore`

The @ Symbol: Your Context Navigator



@file

```
@src/components/Header.tsx
```

Reference specific files



@folder

```
@src/utils/
```

Include entire directories



@codebase

```
@codebase "authentication"
```

Semantic search



@docs

```
@docs "React hooks"
```

Query documentation



@git

```
@git "last commit"
```

Reference git history



@branch

```
@branch feature/new-auth
```

Access other branches

Effective Prompting



Make this better

- No success criteria
- No direction
- Ambiguous goal



`@src/components/Button.tsx`

Refactor to:

1. Add TypeScript props
2. Include disabled state
3. Add loading spinner
4. Keyboard accessible



Describe the diff you want: current state → desired state

Project Rules: .cursor/rules

Four Application Modes

1. **Always Apply** - Every session automatically
2. **Apply Intelligently** - AI determines relevance (recommended)
3. **Apply to Specific Files** - Pattern matching (*.test.ts)
4. **Apply Manually** - Activated with @-mention

Project Rules: .cursor/rules

Four Application Modes

1. **Always Apply** - Every session automatically
2. **Apply Intelligently** - AI determines relevance (recommended)
3. **Apply to Specific Files** - Pattern matching (*.test.ts)
4. **Apply Manually** - Activated with @-mention



When to Use

- Recurring mistakes
- Critical security patterns
- Team consistency

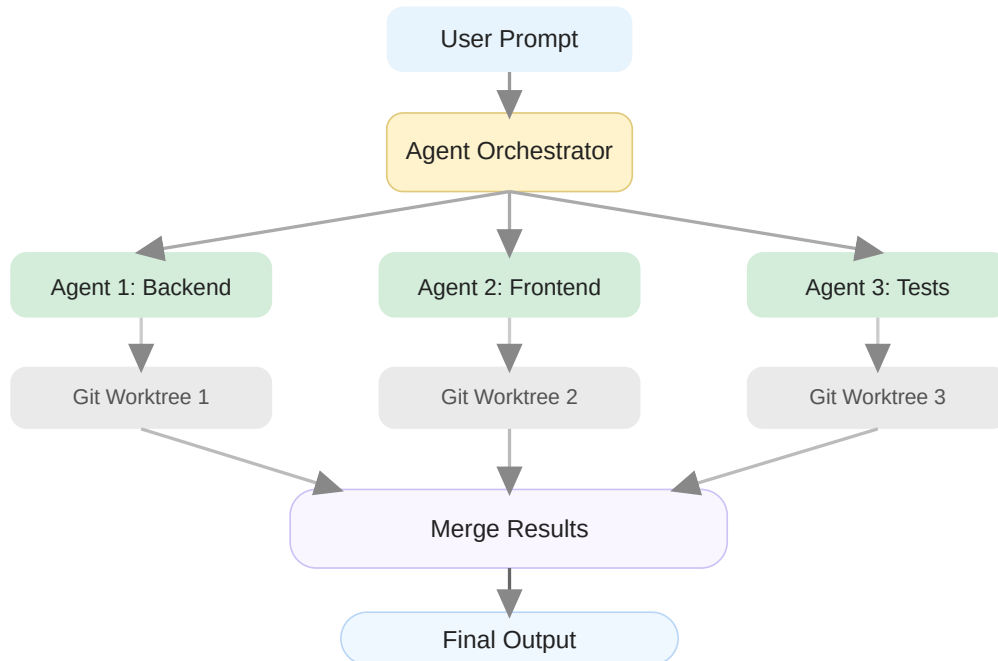


Watch Out

- Can slow you down
- Keep under 500 lines
- Better context > more rules

^[1][Cursor for Staff Engineers - Rules Discussion](#)

Multi-Agent Architecture



Each agent operates in isolated git worktree → No conflicts

Cursor now ships this as **Projects** (Sept 2026, beta): a coordinator agent plans and delegates to parallel subagents, each on its own cloud VM — closing your laptop doesn't stop it

^[1][Cursor Projects](#)

What Is a Git Worktree?

One clone, several working folders — each checked out to a different branch, all sharing the same `.git` history

Branch switching

```
git checkout feature/auth
```

- One folder, one branch at a time
- Stash or commit your WIP first
- Reinstall / rebuild after each switch

Worktrees

```
git worktree add ../auth feature/auth
```

- Many folders, many branches, at once
- Nothing to stash — your files never move
- Each folder keeps its own build state



Three agents → three folders → zero file collisions

Review each diff on its own, merge the ones you want, then `git worktree remove ../auth`

Model Context Protocol (MCP)

Open protocol connecting AI to tools & data

Like USB for AI—standardized interface

Model Context Protocol (MCP)

Open protocol connecting AI to tools & data
Like USB for AI—standardized interface



Tools

Functions AI executes



Prompts

Templated workflows



Resources

Structured data sources

GitHub MCP

- Create PRs
- Review code
- Manage issues

```
npx @modelcontextprotocol/server-github
```

Figma MCP

- Extract design tokens
- Generate components
- Sync UI updates

```
bunx cursor-talk-to-figma-mcp
```

GitMCP

- Turn any repo into MCP
- Replace github.com → gitmcp.io
- Instant context access

```
gitmcp.io/facebook/react
```

Database MCPs

- PostgreSQL
- SQLite
- Query and inspect

```
@modelcontextprotocol/server-postgres
```

Demo: Cursor Configuration Setup



See: `presentation/scripts.md`

Demo 1: Setting Up Cursor Configuration

Questions?

Open floor—ask about anything from this section

Break

10 Minutes 

Building Your First Full App with Cursor

AI-Driven Project Initialization



1. Describe

Problem, users, features
in plain English



2. Define

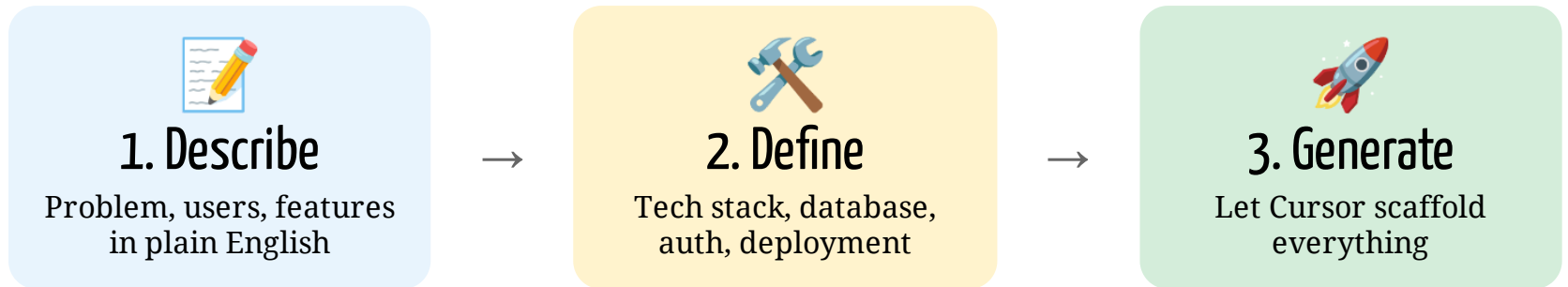
Tech stack, database,
auth, deployment



3. Generate

Let Cursor scaffold
everything

AI-Driven Project Initialization



The Context File Approach

- Create `docs/context.md` with your vision
- Reference it inside prompts: 'Generate the project structure for `@docs/context.md` '

[1] [Building Apps with AI - Complete Workflow](#)

Example: Context File

DeepWork AI - Focus Productivity App

Problem

Developers struggle with multitasking and distractions.
Need a tool to prioritize tasks and maintain deep work focus.

Users

Software engineers, designers, product managers

Core Features

1. Task management with priority levels
2. Focus timer for single-task concentration
3. AI chat to quickly add tasks via natural language

Tech Stack

- Frontend: Next.js 15+ (App Router), TypeScript, Tailwind
- Backend: Supabase (database + auth)
- AI: OpenAI API for chat
- Deployment: Vercel

Database Schema

[Let Cursor generate based on features]

ChatGPT + Cursor Workflow

1 ChatGPT

Planning Phase:

- Brainstorm features
- Structure ideas
- Create context.md
- Plan architecture



2 Cursor

Implementation:

- Generate code
- Multi-file operations
- Debug errors
- Refactor



Use ChatGPT (or Claude/Gemini/etc.) for exploration, Cursor for implementation

Demo: Building Your First Full App

Setting Up Supabase Backend

1. Create Project

- Visit supabase.com
- Create new project
- Copy API URL & anon key

2. Environment Variables

```
NEXT_PUBLIC_SUPABASE_URL=...  
NEXT_PUBLIC_SUPABASE_ANON_KEY=...
```

3. Prompt Cursor

```
@docs/context.md
```

```
Install Supabase client and create  
the database schema for tasks with:  
title, description, priority,  
deadline, completed status, user_id.
```

✓ **Cursor generates SQL migrations
+ RLS policies**

Creating UI Components

Prompt for Component Generation:

Create the following components following our project rules:

1. TaskCard - Display single task with priority badge
2. TaskList - Grid of TaskCards with filtering
3. AddTaskModal - Form to create new tasks
4. FocusTimer - Countdown timer for focused work

Use Tailwind, make them responsive, and include TypeScript interfaces for all props.

✓ Agent Mode Delivers:

- TypeScript types
- Responsive design
- Accessibility
- Smooth animations

💡 Pro Tip:

Work file by file, review each component before moving to the next

Iterative Development Loop

1. Generate code with Agent or Composer



2. Run the app - Check for errors: `npm run dev`



3. Copy error messages - Paste into Cursor chat



4. Let Cursor fix - Review the changes



5. Commit - Save working state with git

^[1][Complete App Building Workflow](#)

Testing with AI Assistance

Generate Test Suites:

@src/components/TaskCard.tsx

Create a complete test suite using Jest and React Testing Library.

Include:

- Rendering tests
- User interaction tests
- Edge cases
- Accessibility tests

Testing with AI Assistance

Generate Test Suites:

```
@src/components/TaskCard.tsx
```

Create a complete test suite using Jest and React Testing Library.

Include:

- Rendering tests
- User interaction tests
- Edge cases
- Accessibility tests



Component Tests



Integration Tests



E2E Tests

Questions?

Open floor—ask about anything from this section

Break

10 Minutes 

Advanced Architecture Patterns



Atomic Design

- **Atoms** - Button, Input
- **Molecules** - SearchBar
- **Organisms** - Header
- **Templates** - Layouts
- **Pages** - Full views



State Management

- **Local** - useState
- **Global Client** - Zustand
- **Server** - React Query
- **URL** - searchParams



Custom Hooks

- Single responsibility
- Named exports
- TypeScript interfaces
- Reusable logic



Performance

- React.memo()
- useMemo()
- useCallback()
- Code splitting

Demo: AI-Powered Quiz App

Quiz App: Tech Stack

AI-Generated Quiz Application



Frontend

Next.js 15+
TypeScript
Tailwind CSS



AI

Anthropic API
claude-sonnet-5
Structured Output



Backend

Supabase
Real-time
Row Level Security

Architecture Diagrams with Mermaid

Prompt Cursor to Generate Diagrams:

Create a Mermaid diagram showing the data flow in our quiz app:

- User selects topic

- API generates questions with OpenAI

- Questions displayed one at a time

- Answers stored in Supabase

- Results calculated and shown

Architecture Diagrams with Mermaid

Prompt Cursor to Generate Diagrams:

Create a Mermaid diagram showing the data flow in our quiz app:

- User selects topic

- API generates questions with OpenAI

- Questions displayed one at a time

- Answers stored in Supabase

- Results calculated and shown

Diagram Types

- flowchart - Logic flow
- sequenceDiagram - Interactions
- classDiagram - Data structures



Best Practice

Start small, layer upward, multiple perspectives

[1][Cursor Cookbook - Mermaid Diagrams](#)

Figma MCP Integration



Figma Design

- Design tokens
- Component specs
- Typography



React Code

- Pixel-perfect
- Tailwind classes
- Responsive

Prompt Example:

```
@figma file:quiz-app-designs
```

```
Generate the QuestionCard component matching the Figma design.  
Extract all design tokens (colors, spacing, shadows) and  
create a matching React component with Tailwind.
```

^[1][Cursor Talk to Figma MCP](#)

Questions?

Open floor—ask about anything from this section

Full-Stack Application

Backend API Development

Prompt for Complete REST API:

Create a complete REST API for our quiz app:

Endpoints:

- POST /api/quiz/generate - Create new quiz
- GET /api/quiz/[id] - Get quiz by ID
- POST /api/quiz/[id]/answer - Submit answer
- GET /api/quiz/[id]/results - Get final results
- GET /api/quiz/history - User's past quizzes

Include: Authentication, validation (Zod), error handling, rate limiting, API documentation comments



Auth



Validation



Rate Limiting

Database Optimization

Optimization Strategies

- **Indexes** - Speed up queries
- **JOINS** - Reduce round trips
- **Pagination** - Limit results
- **Caching** - Redis layer
- **Connection Pooling** - Supervisor

Prompt Cursor:

```
@src/lib/db/queries.ts
```

Optimize this query for performance:

- Add proper indexes
- Use JOIN instead of multiple queries
- Implement pagination
- Add caching strategy

State Management & Real-time



Zustand Store

Quiz state, answers, timer



Optimistic Updates

Instant UI feedback



Real-time Sync

Supabase subscriptions

Optimistic Update Pattern

1. Update UI immediately (optimistic)
2. Send request to server
3. If error: rollback UI changes
4. If success: keep UI as-is

Result: App feels instant ⚡

Demo: Test, Iterate, Deploy

Questions?

Open floor—ask about anything from this section

Break

10 Minutes 

Deployment & Production

Deployment with Vercel

From Code to Production in 10 Minutes



Zero Config

Auto optimizations



Edge Network

Global CDN



Instant Rollback

One-click revert

CI/CD Pipeline

Generate GitHub Actions Workflow:

Create a GitHub Actions workflow for our Next.js app:

On every PR:

1. TypeScript type checking
2. ESLint
3. Tests (unit + integration)
4. Build
5. Deploy preview to Vercel

On merge to main:

All above + E2E tests + Production deploy + Slack notification



Type Check



Test



Deploy

Production Readiness Checklist

Security

- Env vars secured
- SSL + CORS configured
- Rate limiting enabled
- Dependency & secret scanning

Performance

- CDN + edge caching
- Image optimization
- Code splitting
- Database indexes

Code Quality

- Type safety (strict TS)
- ESLint + a11y checks
- 80% test coverage

Monitoring & Recovery

- Error tracking (Sentry)
- Uptime + analytics
- Automated backups
- Rollback strategy

Automated PR Workflow

Staff Engineer Tip: Handle PR Feedback Fast



1. Screenshot

Capture PR comments



2. Cursor Plan

Drag into Cursor,
generate plan



3. Execute

Fix items one by one

Even faster: run `/review` (`/review-bugbot`, `/review-security`) before you push—
Bugbot integrates with GitHub & GitLab PRs

^[1][Automating PR Feedback Workflow](#) | ^[2][Cursor Changelog—/review](#)

Demo: Vercel & Github Actions

Cost Optimization Strategies



Current Costs

- Vercel Pro: \$20/month
- Supabase Pro: \$25/month
- Anthropic API: ~\$50/month
- Upstash Redis: \$10/month

Total: ~\$105/month



Optimizations

- ☒ Cheaper model tier where quality allows (gpt-5.4-nano ~\$0.20/1M)
- ☒ Aggressive caching
- ☒ Free tier for previews
- ☒ Optimize DB queries
- ☒ Rate limiting

New Total: ~\$40/month



Prompt Cursor for Cost Analysis

Analyze our app's costs and suggest optimizations to reduce spending while maintaining performance.

Key Takeaways

✓ Context is King

Strategic @-mentions, clean project structure

✓ Three-Step Framework

Explore → Plan → Build

✓ Production-Ready

Testing, CI/CD, monitoring, security

✓ MCP Integration

GitHub, Figma, databases



Remember

Cursor augments your workflow—it doesn't replace engineering judgment. Use it to eliminate tedious work and focus on what matters.

Additional Resources



Documentation

- [Cursor Docs](#)
- [Changelog](#)
- [Composer 2.5](#)
- [Cursor Projects](#)



MCP Servers

- [Official Servers](#)
- [GitMCP](#)
- [Figma MCP](#)



Videos

- [Staff Engineers](#)
- [Building Apps](#)

Connect With Me



[Course materials](#)



[LinkedIn](#)



[Twitter/X - @LucasEnkrateia](#)



[YouTube - @automatalearninglab](#)



[Blog](#)



Email: lucasenkrateia@gmail.com